



**SCHOOL OF MATHEMATICS AND APPLIED SCIENCES
DEPARTMENT OF COMPUTING**

TO : MR. S. YUTE

FROM : DEBORAH CHIRAMBO & SHALOM MAKWINJA

REGISTRATION NO : BSC-COM-NE-19-21 & BSC-COM-NE-19-22

COURSE NAME : NETWORK PROGRAMMING AND APPLICATION TO DEVELOPMENT

COURSE CODE : NET322

ASSIGNMENT TITLE : THEORETICAL FOUNDATIONS OF ASYNCIO-BASED IOT TELEMETRY SYSTEM

DUE DATE : 8th May, 2026

THEORETICAL FOUNDATIONS OF ASYNCIO-BASED IOT TELEMETRY SYSTEM

A. Serialization Choices

Different serialization formats are used across the telemetry system because each communication layer has distinct requirements in terms of performance, readability, and interoperability. YAML is used for configuration, protobuf is used for server to server communication, and JSON/XML/YAML are supported at the REST API layer.

YAML is chosen for configuration because it is highly human-readable and easy to modify without requiring changes to source code. Sensor definitions such as identifiers, types, and reporting intervals are represented using clean indentation-based structures, making it more readable than JSON's bracket-heavy syntax. Since configuration files are loaded infrequently, performance overhead is not a major concern; instead, readability and maintainability are prioritized.

Protobuf is used for sensor-to-server communication due to its efficiency in IoT environments. It encodes data in a compact binary format, significantly reducing message size and bandwidth usage when multiple sensors transmit frequent updates. In addition, Protobuf enforces a strict schema through the definition file, ensuring both sender and receiver share a consistent data structure. This reduces runtime errors by detecting mismatches at the schema level.

At the REST API layer, JSON, XML, and YAML are supported to ensure interoperability with different client types. JSON is lightweight and widely supported in web and mobile applications, making it ideal for modern clients. XML is retained for compatibility with legacy systems that depend on structured document formats. YAML support is included for scripting and configuration-based clients. All formats are served through the same endpoints using HTTP content negotiation, eliminating the need for separate APIs per format.

B. Web Services Protocol

REST is well-suited for the historical telemetry API because the system is naturally resource-oriented. Core entities such as sensors and sensor readings can be directly modeled as resources accessible through URLs like and these resources are manipulated using standard HTTP methods such as GET, POST, and DELETE, resulting in a simple and intuitive interface.

A key advantage of REST is its statelessness. Each request contains all the necessary information for processing, meaning the server does not maintain session state for individual clients. This improves scalability, as multiple clients can independently query historical data without requiring server-side session tracking.

In contrast, SOAP introduces unnecessary complexity through XML-based envelopes and strict messaging standards, making it unsuitable for lightweight telemetry systems. RPC-based approaches are also less appropriate because they are operation-oriented rather than resource-oriented, leading to tighter coupling between client and server. REST provides a more flexible and scalable architecture aligned with modern web systems.

C. AsyncIO vs Threading

The telemetry system is primarily I/O-bound, as most of its time is spent waiting for external operations such as sensor communication, HTTP requests, database interactions, and WebSocket message delivery. As a result, AsyncIO is a more efficient concurrency model than traditional threading.

AsyncIO uses an event loop and cooperative multitasking through coroutines. When a coroutine encounters an expression, it yields control to the event loop, allowing other tasks to execute. This enables a single-threaded server to manage thousands of concurrent connections efficiently without the overhead of creating one thread per connection.

Compared to threading, AsyncIO consumes less memory and avoids synchronization issues such as race conditions and deadlocks, since coroutines execute in a controlled event loop rather than concurrently across multiple threads. Threading may still be useful when integrating blocking third-party libraries, while multiprocessing is better suited for CPU-intensive workloads such as large-scale data processing. However, for a network-heavy system like this telemetry pipeline, AsyncIO is the most appropriate choice.

D. HTTP Mechanics

Content negotiation allows a single REST endpoint to return different response formats depending on the client's header. Clients may request, or/and the server responds with the appropriate serialization format.

This mechanism enables the system to support browsers, scripts, and legacy applications without duplicating API endpoints or logic.

The WebSocket upgrade handshake enables real-time communication between the server and dashboard clients. A WebSocket connection begins as a standard HTTP request containing headers such as and if accepted by the server, it responds with a status, upgrading the connection to a persistent full-duplex channel. This allows the server to push sensor updates instantly as they arrive, eliminating the need for client-side polling and reducing communication latency.

References

1. IETF. (2011). *RFC 6455: The WebSocket protocol*. <https://datatracker.ietf.org/doc/html/rfc6455>
2. IETF. (2014). *RFC 7231: Hypertext transfer protocol (HTTP/1.1): Semantics and content*. <https://datatracker.ietf.org/doc/html/rfc7231>
3. Google. (n.d.). *Protocol buffers documentation*. <https://protobuf.dev/>
4. YAML.org. (n.d.). *YAML Ain't Markup Language (YAML) version 1.2 specification*. <https://yaml.org/spec/>
5. Richardson, L., & Ruby, S. (2007). *RESTful web services*. O'Reilly Media. <https://www.oreilly.com/library/view/restful-web-services/9780596529260/>
6. Python Software Foundation. (n.d.). *asyncio — Asynchronous I/O*. <https://docs.python.org/3/library/asyncio.html>
7. Kleppmann, M. (2017). *Designing data-intensive applications*. O'Reilly Media. <https://dataintensive.net/>